Equity — metamodel pipeline

This notebook assembles the end-to-end metamodel for the three equity index futures — the S&P 500, Nasdaq-100 and Euro Stoxx 50 e-minis. The primary signal behaves as a short-horizon contrarian rule across the index complex: positions entered the day after a signal earn a positive forward return, yet they lean against the preceding day's move. The metamodel's job is narrower than the signal's — it predicts whether a given trigger is worth taking. Among the three, the Nasdaq carries the cleanest edge, the S&P is suggestive but rests on a thin single-shot test, and the Euro Stoxx fails to clear chance out of sample. The sections below trace that conclusion from raw prices through to the instruments' contribution to the pooled book.

Universe. `es1s`, `nq1s`, `fesx1s` (3 contracts).

Reading guide. The notebook runs top to bottom against committed artifacts; no model is retrained. Each stage loads its result files and embeds the figures already produced by the pipeline. Section order mirrors the marking scheme in `reports/harry/00-context.md`: data, signal, labelling, features, model selection, cluster importance, evaluation.

Source of truth. Where a number appears it is loaded from a file in this cell's output, not transcribed from prose; the few places where the committed data and the written reports disagree are flagged inline.

In [1]:
```
CLASS = 'equity'
INSTS = ['es1s', 'nq1s', 'fesx1s']
NAMES = {'es1s': 'S&P 500 e-mini (ES)', 'nq1s': 'Nasdaq-100 e-mini (NQ)', 'f

from pathlib import Path
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from IPython.display import Image, display, Markdown


def _repo_root() -> Path:
    here = Path.cwd().resolve()
    for q in [here, *here.parents]:
        if (q / "pyproject.toml").exists():
            return q
    return here


REPO = _repo_root()
OUT = REPO / "src/stml/new_work/outputs"
RESULTS = REPO / "results"
REPORTS = REPO / "reports"
```

```python
DATA = REPO / "data"


def show(path, width=1000):
    p = Path(path)
    if p.exists():
        display(Image(str(p), width=width))
    else:
        print(f"[missing figure] {p}")


def load(path) -> pd.DataFrame:
    p = Path(path)
    if p.exists():
        return pd.read_csv(p)
    print(f"[missing data] {p}")
    return pd.DataFrame()


def table(df, n=None, caption=None):
    if df is None or len(df) == 0:
        print("[empty table]")
        return
    d = df.head(n) if n else df
    sty = d.style.format(precision=4).hide(axis="index")
    if caption:
        sty = sty.set_caption(caption)
    display(sty)
```

1. Data and cleaning

The price history runs from 1990, while the primary signal is released only over 2020-01-03 to 2022-06-30; the sealed test period sits beyond the embargo. Loading goes through `stml.io`, and the missing-data policy — keep zero-volume weekday settles, drop weekend and out-of-bounds rows — is documented in `reports/missing-data-report.md` and implemented in `stml.na_checks`. The coverage table below is computed directly from `data/ohlcv_data.csv` for this class's contracts.

In [2]:
```python
ohlcv = load(DATA / "ohlcv_data.csv")
sub = ohlcv[ohlcv["instrument"].isin(INSTS)]
cov = (sub.groupby("instrument")
          .agg(first_date=("date", "min"), last_date=("date", "max"), n_rows
          .reindex(INSTS).reset_index())
table(cov, caption=f"{CLASS.title()} — OHLCV coverage (data/ohlcv_data.csv)"
```

Equity — OHLCV coverage (data/ohlcv_data.csv)

instrument	first_date	last_date	n_rows
es1s	1997-09-09	2022-06-30	6296
nq1s	1999-06-21	2022-06-30	5845
fesx1s	1998-06-30	2022-06-30	6108

2. Signal characterisation

All three indices show a positive next-day relationship between the signal and the realised return, which is what makes the trigger tradeable at a one-day lag. The contrarian character is equally clear: the signal loads negatively on the trailing one-day move, so it fires after pull-backs rather than with momentum. The Nasdaq and S&P express this most strongly; the Euro Stoxx is the weakest of the three and, as the model section confirms, its edge does not survive an honest hold-out. Read the per-instrument correlations below as a description of the signal's construction, not a promise about the metamodel.

In [3]:

```
sd = load(RESULTS / "harry/signal_direction.csv")
if not sd.empty:
    want = ["instrument", "n_bets", "corr_fwd_1", "corr_fwd_5", "corr_trail_
    cols = [c for c in want if c in sd.columns]
    rows = sd[sd["instrument"].isin(INSTS)][cols].set_index("instrument").re
    table(rows, caption=f"{CLASS.title()} — signal direction (results/harry/
```

Equity — signal direction (results/harry/signal_direction.csv)

instrument	n_bets	corr_fwd_1	corr_fwd_5	corr_trail_1	corr_contemp_0
es1s	575	0.1029	-0.0305	-0.1610	-0.1399
nq1s	604	0.1137	0.0091	-0.0533	-0.1656
fesx1s	637	0.0474	-0.0287	-0.1930	-0.1788

3. Labelling

Events use the triple-barrier method of López de Prado (2018, ch. 3) with entry at `t+1`: the signal is observed at the close of bar `t` and the position is opened the following session, which is justified by the positive next-day correlation shown above. Profit-taking and stop-loss barriers are set in units of a causal volatility estimate, a vertical barrier caps the holding period, and overlapping events are down-weighted by average uniqueness (López de Prado, 2018, ch. 4). The methodology is set out in `reports/harry/02-labels.md`.

The summary below is computed from `results/harry/events.csv` (the event table that feeds the feature and model stages).

In [4]:
```
ev = load(RESULTS / "harry/events.csv")
ev = ev[ev["instrument"].isin(INSTS)]
if not ev.empty:
    g = (ev.groupby("instrument")
           .agg(n_events=("label", "size"),
                label1_share=("label", "mean"),
                mean_uniqueness=("uniqueness_weight", "mean"),
                mean_sigma=("sigma", "mean"))
           .reindex(INSTS).reset_index())
    table(g, caption=f"{CLASS.title()} — triple-barrier events (results/harr
    print(f"Total events for {CLASS}: {len(ev):,}  |  label==1 share: {ev['l
```

Equity — triple-barrier events (results/harry/events.csv)

instrument	n_events	label1_share	mean_uniqueness	mean_sigma
es1s	564	0.5780	0.1286	0.0133
nq1s	593	0.5970	0.1209	0.0163
fesx1s	626	0.5160	0.1137	0.0145

```
Total events for equity: 1,783  |  label==1 share: 0.563
```

4. Features

Predictors are grouped into causal families, all computed at the signal date and predictive of the next-day label, so none peeks into the future. The micro families (`reports/harry/03-features.md`) cover the signal's own trajectory, conditional risk, information-theoretic dependence, fixed microstructure proxies, cross-asset structure, an optional wavelet decomposition and a concept-drift discriminator. A macro block of thirty-one rolling, standardised series (`reports/harry/03-6-macro-features.md`) adds volatility, rates, credit, dollar, commodity-fundamental and growth context. Two hidden-Markov regime blocks — per-instrument volatility (`hmm_vol`) and a global risk-on/risk-off state (`hmm_macro`) — are fit only on pre-sample data and then frozen, so they are train-fitted rather than leaking. Features are tagged `E` (purely causal) or `TF` (train-fitted on pre-sample) accordingly.

In [5]:
```
# The feature matrix is built by the pipeline; here we confirm the frozen HM
# regime features are present for this class's instruments.
for fn in ["features_hmm_vol.csv", "features_hmm_macro.csv"]:
    f = load(REPO / "src/stml/new_work" / fn)
    if not f.empty and "instrument" in f.columns:
        n = f[f["instrument"].isin(INSTS)].shape[0]
        print(f"{fn}: {n:,} rows for {CLASS} instruments | columns: {list(f.
    elif not f.empty:
        print(f"{fn}: {f.shape[0]:,} rows | columns: {list(f.columns)}")
```

```
features_hmm_vol.csv: 1,895 rows for equity instruments | columns: ['date', 'instrument', 'hmm_vol_p0_calm', 'hmm_vol_p2_turbulent', 'hmm_vol_next_turbulent', 'hmm_vol_entropy']
features_hmm_macro.csv: 1,935 rows for equity instruments | columns: ['date', 'instrument', 'hmm_macro_p0', 'hmm_macro_next_riskoff', 'hmm_macro_entropy']
```

5. Model selection

Each instrument runs a horse race across penalised logistic regression, a random forest, gradient boosting and a small multi-task network, scored by combinatorial purged cross-validation (six groups, two held out, fifteen paths) with per-instrument purging and embargo. Feature variants (full, pruned, reduced) are compared, and the simplest variant within one cross-path standard error of the best is locked **before** the out-of-sample window is touched. The cross-validation runs on the training partition only; the sealed test (after 2021-10-20) is scored once, with the locked choice fixed.

The horse race lands on a different architecture for each index, which is itself informative — there is no single equity model. A penalised logistic fit wins for the S&P, gradient boosting for the Nasdaq, and a random forest for the Euro Stoxx. The first two clear their lower confidence bound and are flagged as carrying signal; the Euro Stoxx forest sits below the 0.5 line out of sample and is not. Note the gap between the cross-validated and single-shot numbers for the S&P: a respectable development score paired with a thin test set is a hypothesis, not a verdict.

In [6]:
```python
mc = OUT / "model_comparison" / CLASS
sel = load(OUT / "model_comparison/selection_table.csv")
locked = load(mc / "locked_picks.csv")
cpcv = load(mc / "cpcv_results.csv")
oos = load(mc / "oos_results.csv")

if not sel.empty:
    keep = [c for c in ["instrument", "best_model", "best_auc", "lower_ci",
            if c in sel.columns]
    sc = sel[sel["instrument"].isin(INSTS)][keep].set_index("instrument").re
    table(sc, caption=f"{CLASS.title()} — champion model and signal verdict

if not locked.empty:
    table(locked, caption="Locked feature variant per instrument (1SE rule,

for inst in INSTS:
    display(Markdown(f"**{NAMES[inst]} — `{inst}` cross-validated variants**
    show(mc / f"{inst}_cpcv_chart.png", width=760)

show(mc / "oos_summary_chart.png", width=820)

# Out-of-sample AUC for the locked variants only.
if not oos.empty and not locked.empty:
    lk = dict(zip(locked["inst"], locked["locked_variant"]))
    mask = oos.apply(lambda r: lk.get(r["inst"]) == r["variant"], axis=1)
```

```
    keep = [c for c in ["inst", "variant", "auc", "auc_ci_lo", "auc_ci_hi",
    table(oos[mask][keep], caption="Out-of-sample AUC — locked variants only
```

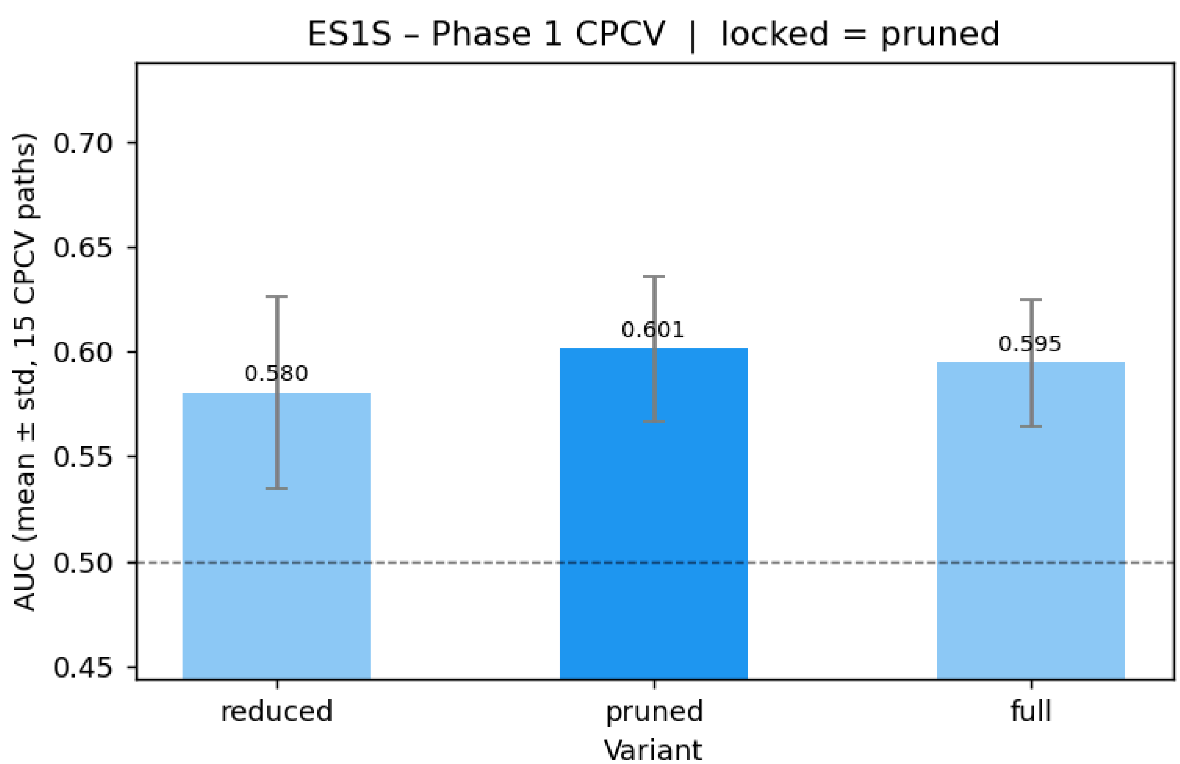
Equity — champion model and signal verdict (selection_table.csv)

instrument	best_model	best_auc	lower_ci	signal	n_events
es1s	logistic	0.5953	0.5668	True	1925
nq1s	xgb	0.5702	0.5374	True	2010
fesx1s	rf	0.4964	0.4722	False	2145

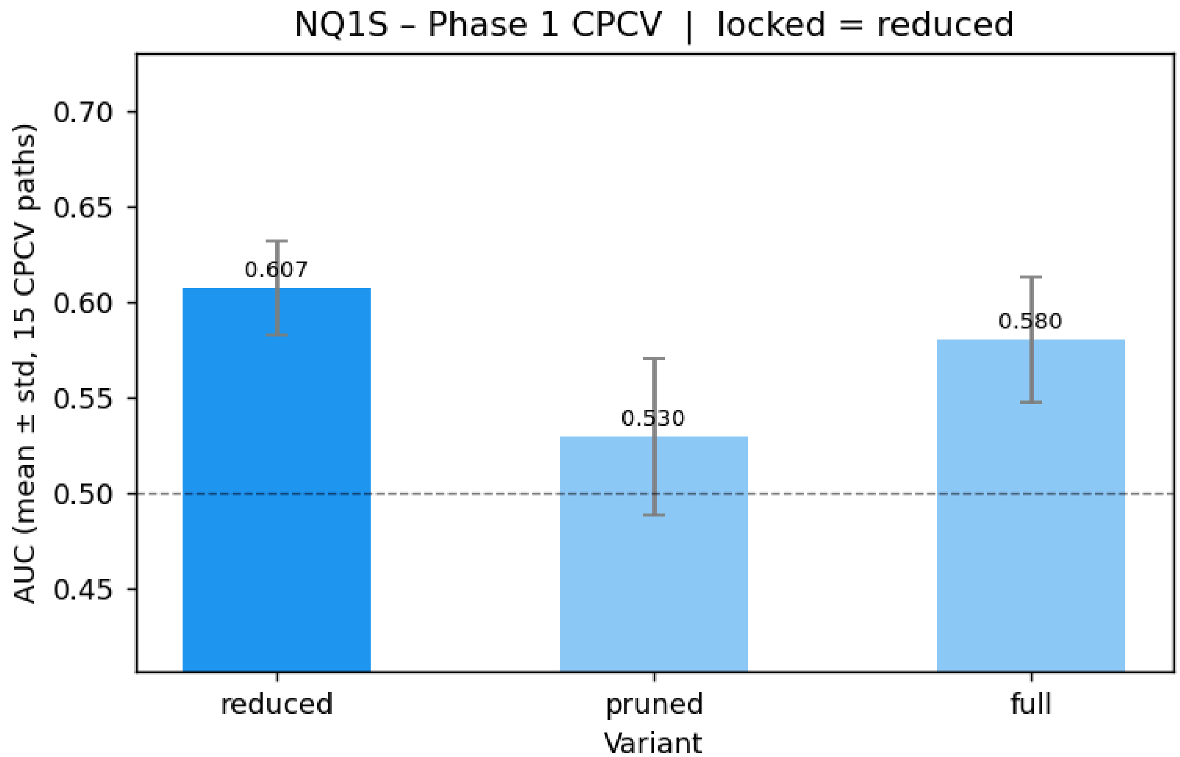
Locked feature variant per instrument (1SE rule, train-only)

inst	locked_variant
nq1s	reduced
fesx1s	reduced
es1s	pruned

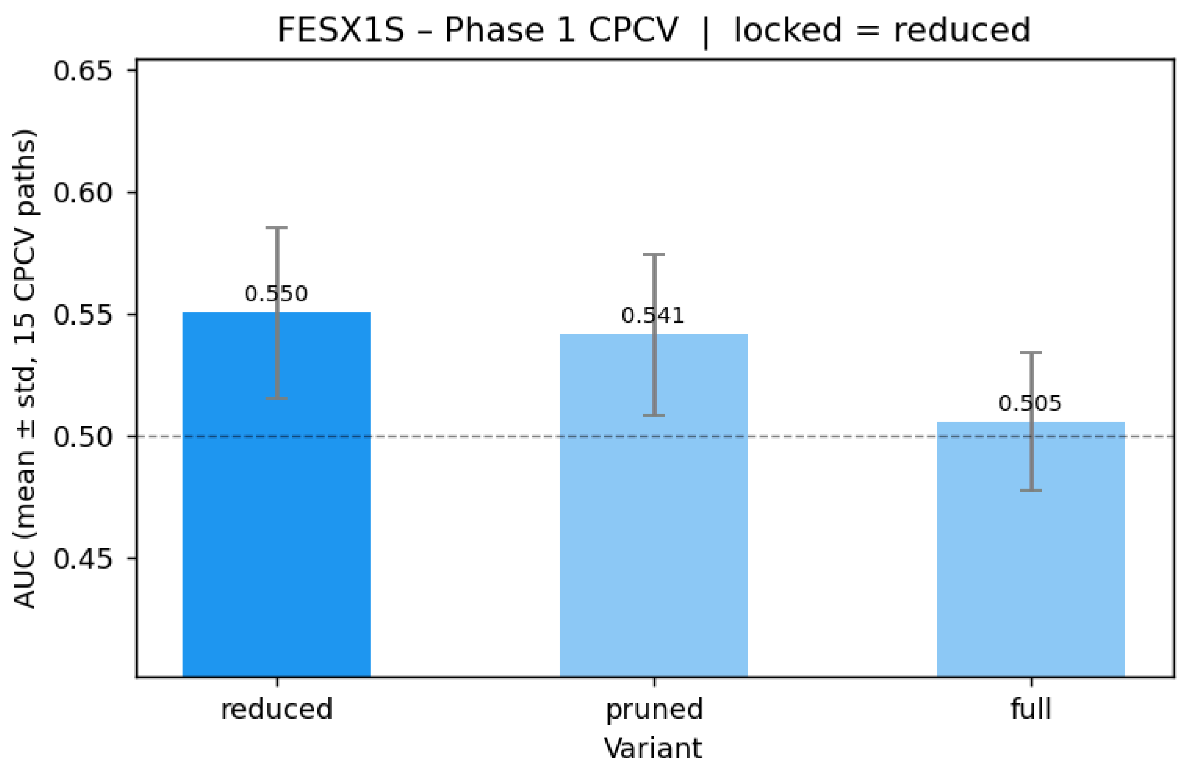
S&P 500 e-mini (ES) — `es1s` cross-validated variants

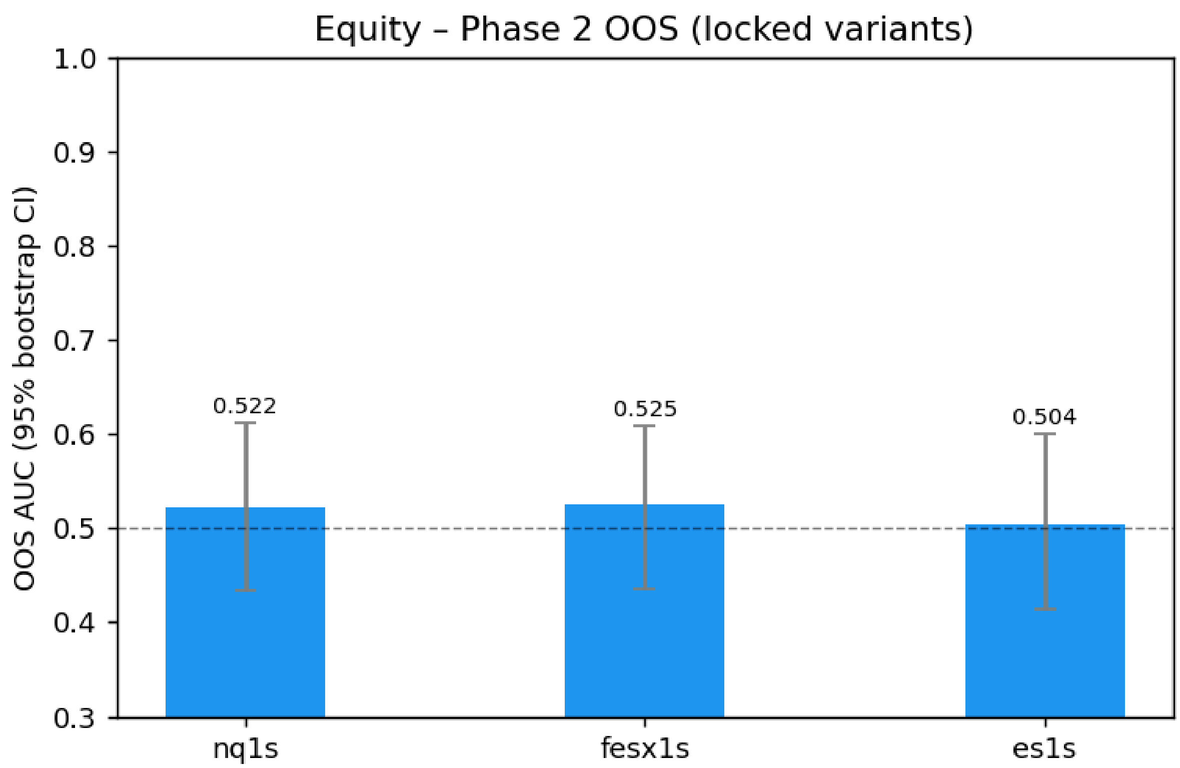


Nasdaq-100 e-mini (NQ) — `nq1s` cross-validated variants



Euro Stoxx 50 (FESX) — `fesx1s` cross-validated variants





Out-of-sample AUC — locked variants only

inst	variant	auc	auc_ci_lo	auc_ci_hi	n_test
nq1s	reduced	0.5215	0.4337	0.6119	171
fesx1s	reduced	0.5248	0.4361	0.6087	176
es1s	pruned	0.5044	0.4147	0.6009	150

6. Feature importance

Importance is read at the cluster level rather than per raw feature, because the predictors are correlated in blocks and per-feature scores would split credit arbitrarily across substitutes. Features are clustered on a rank-correlation distance; the dendrogram shows the block structure, mean-decrease-accuracy gives each cluster's drop, and a global SHAP or coefficient ranking resolves the within-cluster detail. Logistic champions are read through their coefficients, tree and network champions through SHAP.

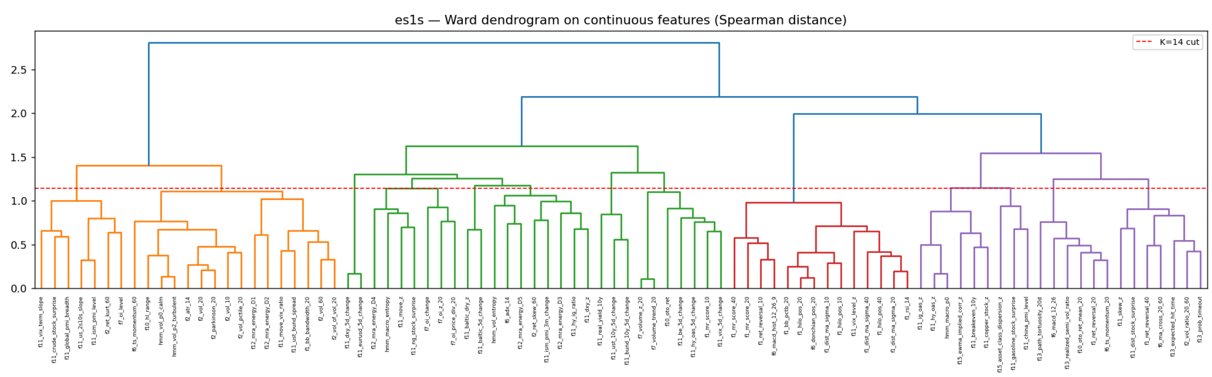
Clustering the features before ranking them matters here because the equity predictors are correlated in blocks — momentum and range measures move together, as do the macro volatility and credit gauges. The cluster-level drops show the signal's own trajectory features and the volatility-regime block doing most of the work for the Nasdaq and S&P. For the Euro Stoxx the importance picture is diffuse, consistent with a model that has found little to hold on to.

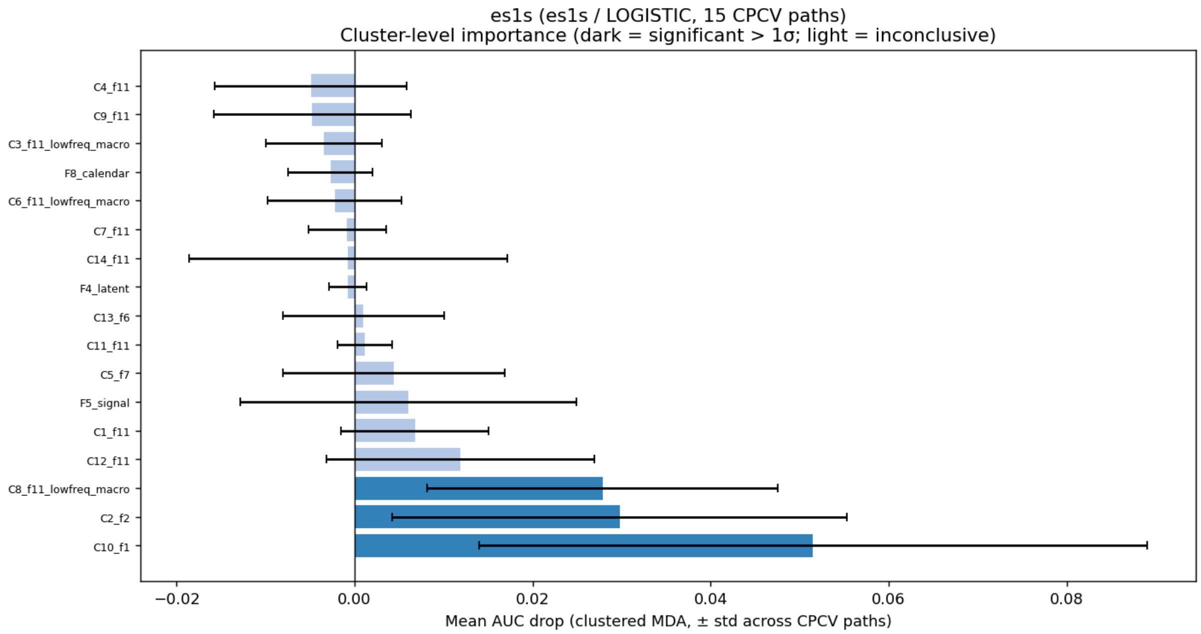
In [7]:

```python
imp = OUT / "importance"
sel = load(OUT / "model_comparison/selection_table.csv")
model_of = dict(zip(sel.get("instrument", []), sel.get("best_model", []))) i

for inst in INSTS:
    d = imp / inst
    display(Markdown(f"### {NAMES[inst]} — `{inst}`"))
    cm = load(d / "champion_meta.csv")
    if not cm.empty:
        table(cm)
    show(d / "dendrogram.png", width=900)
    show(d / "clustered_mda_chart.png", width=820)
    mda = load(d / "clustered_mda_full.csv")
    if not mda.empty:
        table(mda.head(8), caption="Top clusters by mean decrease in accurac
    coef_png = d / "global_coef_chart.png"
    if str(model_of.get(inst, "")) == "logistic" and coef_png.exists():
        show(coef_png, width=820)
        table(load(d / "global_coef_summary.csv").head(15), caption="Top log
    else:
        show(d / "global_shap_chart.png", width=820)
        table(load(d / "global_shap_summary.csv").head(15), caption="Top fea
```

S&P 500 e-mini (ES) — `es1s`

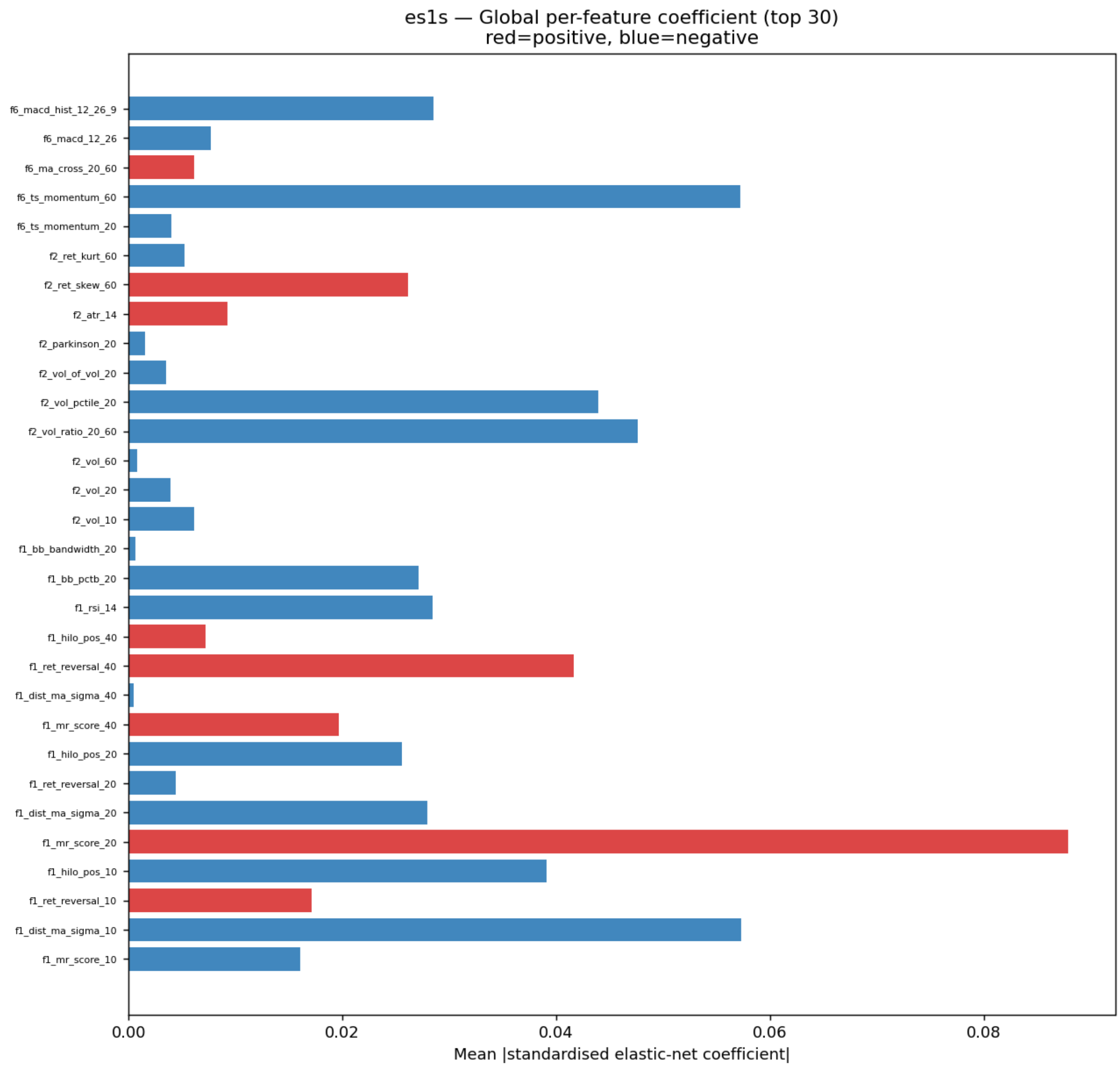
instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	sigi
es1s	equity	es1s	logistic	logistic	0.5953	0.0285	0.5668	Tı





Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C10_f1	0.0515	0.0376	True
C2_f2	0.0298	0.0256	True
C8_f11_lowfreq_macro	0.0279	0.0197	True
C12_f11	0.0119	0.0151	False
C1_f11	0.0068	0.0083	False
F5_signal	0.0060	0.0189	False
C5_f7	0.0044	0.0124	False
C11_f11	0.0011	0.0031	False



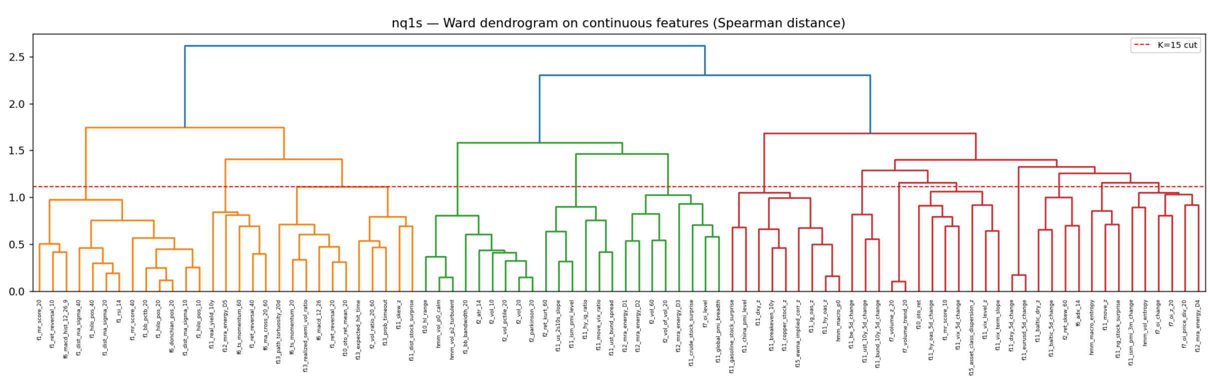
es1s — Global per-feature coefficient (top 30)
red=positive, blue=negative
f6_macd_hist_12_26_9
f6_macd_12_26
f6_ma_cross_20_60
f6_ts_momentum_60
f6_ts_momentum_20
f2_ret_kurt_60
f2_ret_skew_60
f2_atr_14
f2_parkinson_20
f2_vol_of_vol_20
f2_vol_pctile_20
f2_vol_ratio_20_60
f2_vol_60
f2_vol_20
f2_vol_10
f1_bb_bandwidth_20
f1_bb_pctb_20
f1_rsi_14
f1_hilo_pos_40
f1_ret_reversal_40
f1_dist_ma_sigma_40
f1_mr_score_40
f1_hilo_pos_20
f1_ret_reversal_20
f1_dist_ma_sigma_20
f1_mr_score_20
f1_hilo_pos_10
f1_ret_reversal_10
f1_dist_ma_sigma_10
f1_mr_score_10
0.00
0.02
0.04
0.06
0.08
Mean |standardised elastic-net coefficient|

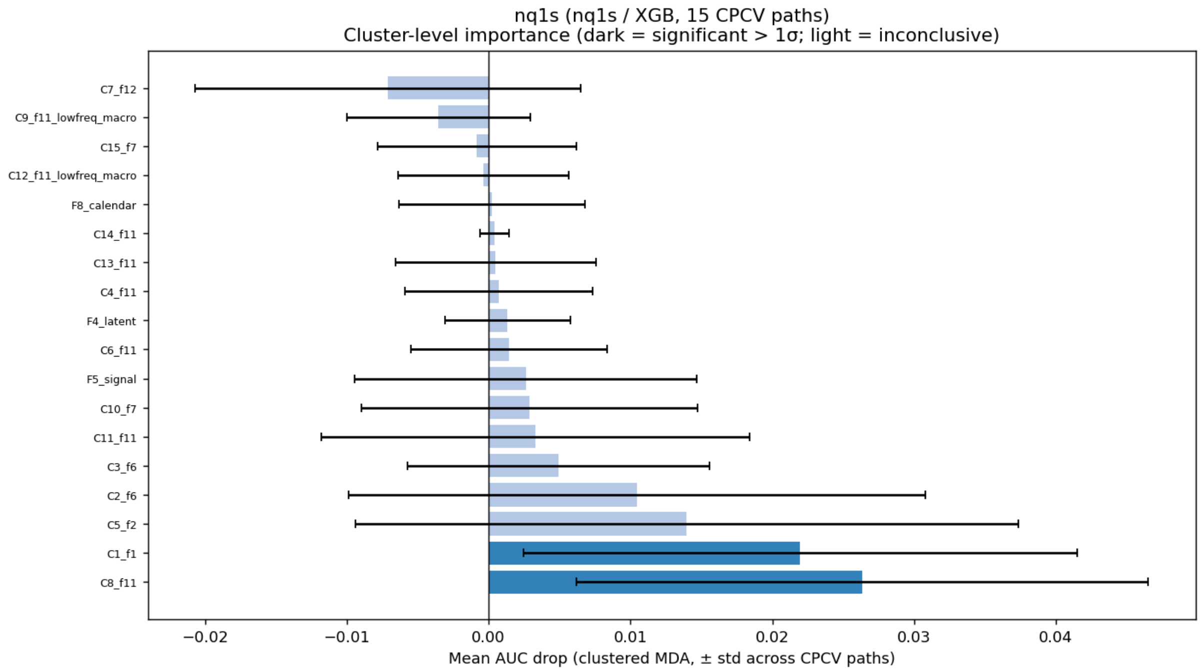
Top logistic coefficients

feature	coef_abs	coef_signed
f11_ust_10y_5d_change	0.2240	-0.2240
f11_gasoline_stock_surprise	0.1081	0.1081
hmm_vol_p2_turbulent	0.1018	0.1018
hmm_vol_p0_calm	0.0925	-0.0925
f1_mr_score_20	0.0879	0.0879
f7_oi_z_20	0.0763	0.0763
f11_hy_oas_5d_change	0.0724	-0.0724
f5_signal	0.0618	0.0618
f11_vix_5d_change	0.0608	-0.0608
f13_path_tortuosity_20d	0.0599	-0.0599
f1_dist_ma_sigma_10	0.0573	-0.0573
f6_ts_momentum_60	0.0572	-0.0572
f11_move_vix_ratio	0.0565	-0.0565
f7_oi_level	0.0515	0.0515
f15_asset_class_dispersion_z	0.0510	0.0510

Nasdaq-100 e-mini (NQ) — `nq1s`

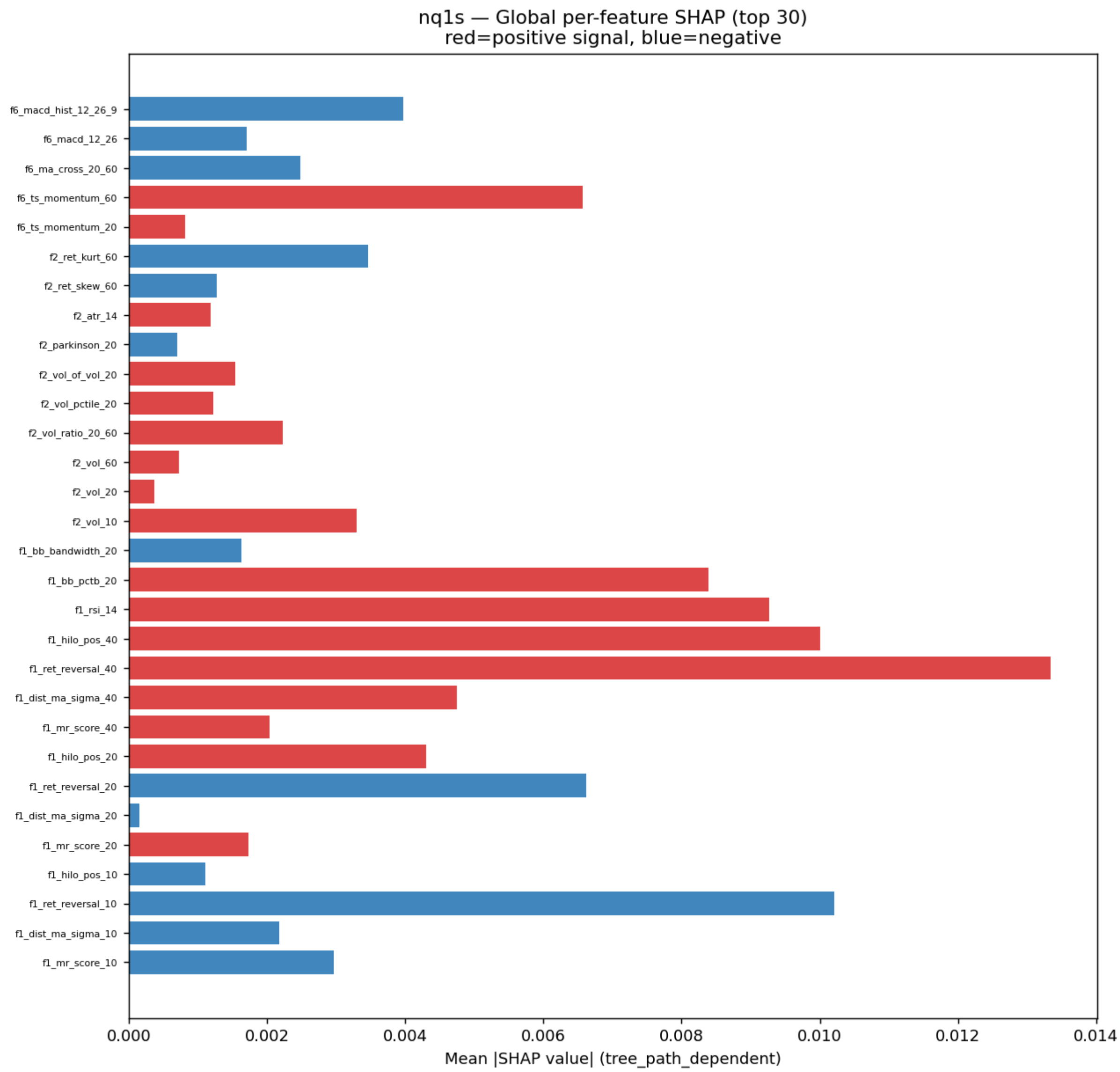
instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	sign
nq1s	equity	nq1s	tree	xgb	0.5702	0.0328	0.5374	Tr





Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C8_f11	0.0263	0.0201	True
C1_f1	0.0220	0.0195	True
C5_f2	0.0140	0.0234	False
C2_f6	0.0105	0.0203	False
C3_f6	0.0049	0.0106	False
C11_f11	0.0033	0.0151	False
C10_f7	0.0029	0.0119	False
F5_signal	0.0026	0.0121	False

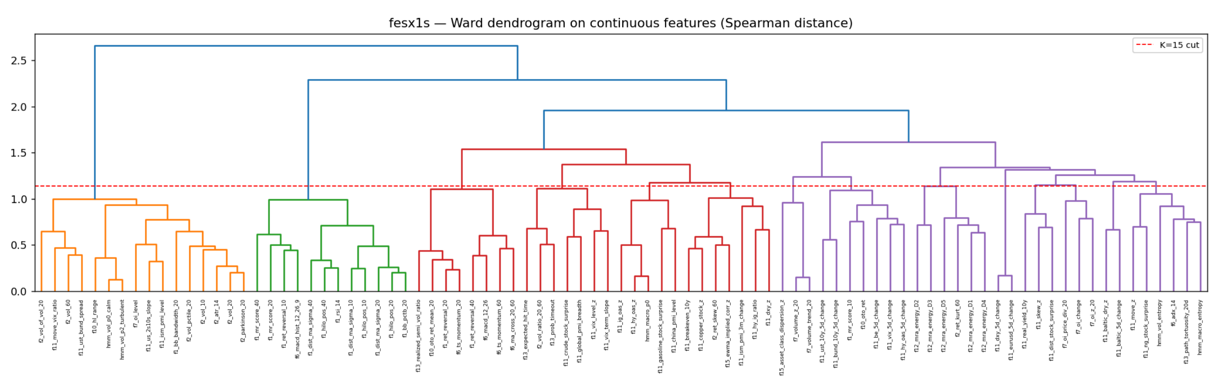

nq1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative
f6_macd_hist_12_26_9
f6_macd_12_26
f6_ma_cross_20_60
f6_ts_momentum_60
f6_ts_momentum_20
f2_ret_kurt_60
f2_ret_skew_60
f2_atr_14
f2_parkinson_20
f2_vol_of_vol_20
f2_vol_pctile_20
f2_vol_ratio_20_60
f2_vol_60
f2_vol_20
f2_vol_10
f1_bb_bandwidth_20
f1_bb_pctb_20
f1_rsi_14
f1_hilo_pos_40
f1_ret_reversal_40
f1_dist_ma_sigma_40
f1_mr_score_40
f1_hilo_pos_20
f1_ret_reversal_20
f1_dist_ma_sigma_20
f1_mr_score_20
f1_hilo_pos_10
f1_ret_reversal_10
f1_dist_ma_sigma_10
f1_mr_score_10
0.000
0.002
0.004
0.006
0.008
0.010
0.012
0.014
Mean |SHAP value| (tree_path_dependent)

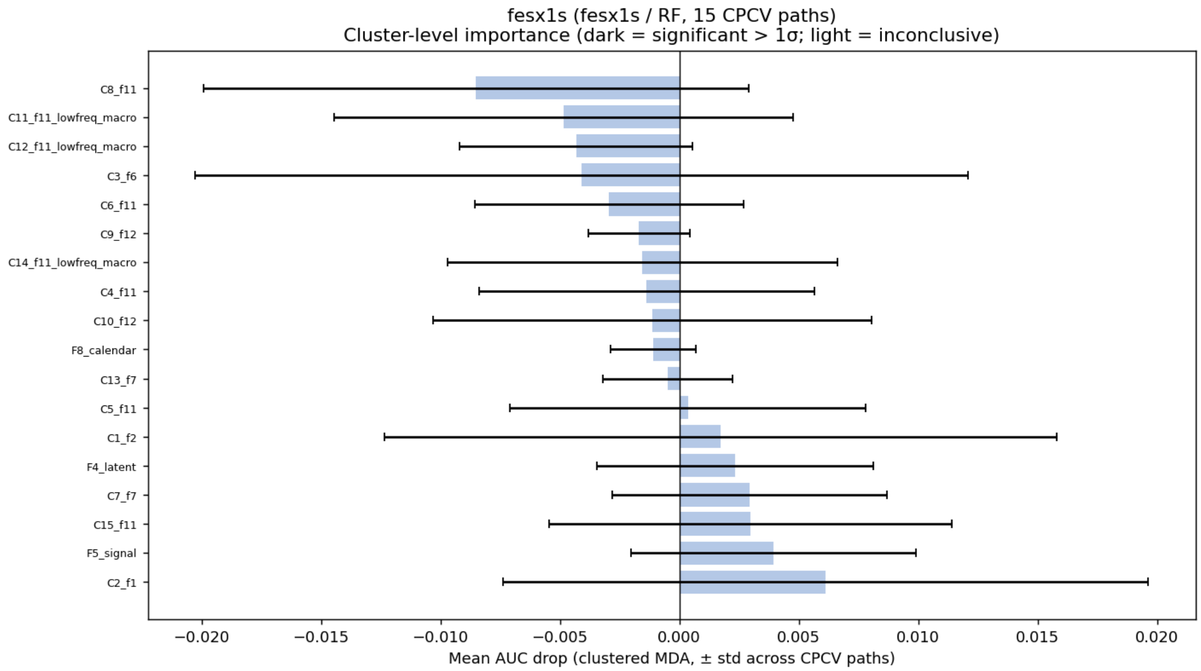
Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
f11_copper_stock_z	0.0501	0.0099	0.0244
f10_hl_range	0.0257	-0.0116	0.0251
f11_real_yield_10y	0.0151	0.0049	0.0114
f10_oto_ret	0.0136	0.0002	0.0171
f15_asset_class_dispersion_z	0.0134	0.0028	0.0147
f1_ret_reversal_40	0.0133	0.0024	0.0192
f11_breakeven_10y	0.0127	0.0055	0.0080
f7_volume_trend_20	0.0125	-0.0015	0.0170
f12_mra_energy_D5	0.0108	-0.0005	0.0164
f1_ret_reversal_10	0.0102	-0.0017	0.0178
f1_hilo_pos_40	0.0100	0.0004	0.0202
f6_adx_14	0.0097	-0.0009	0.0153
f1_rsi_14	0.0093	0.0046	0.0116
f1_bb_pctb_20	0.0084	0.0005	0.0228
f7_oi_change	0.0082	0.0004	0.0156

Euro Stoxx 50 (FESX) — `fesx1s`

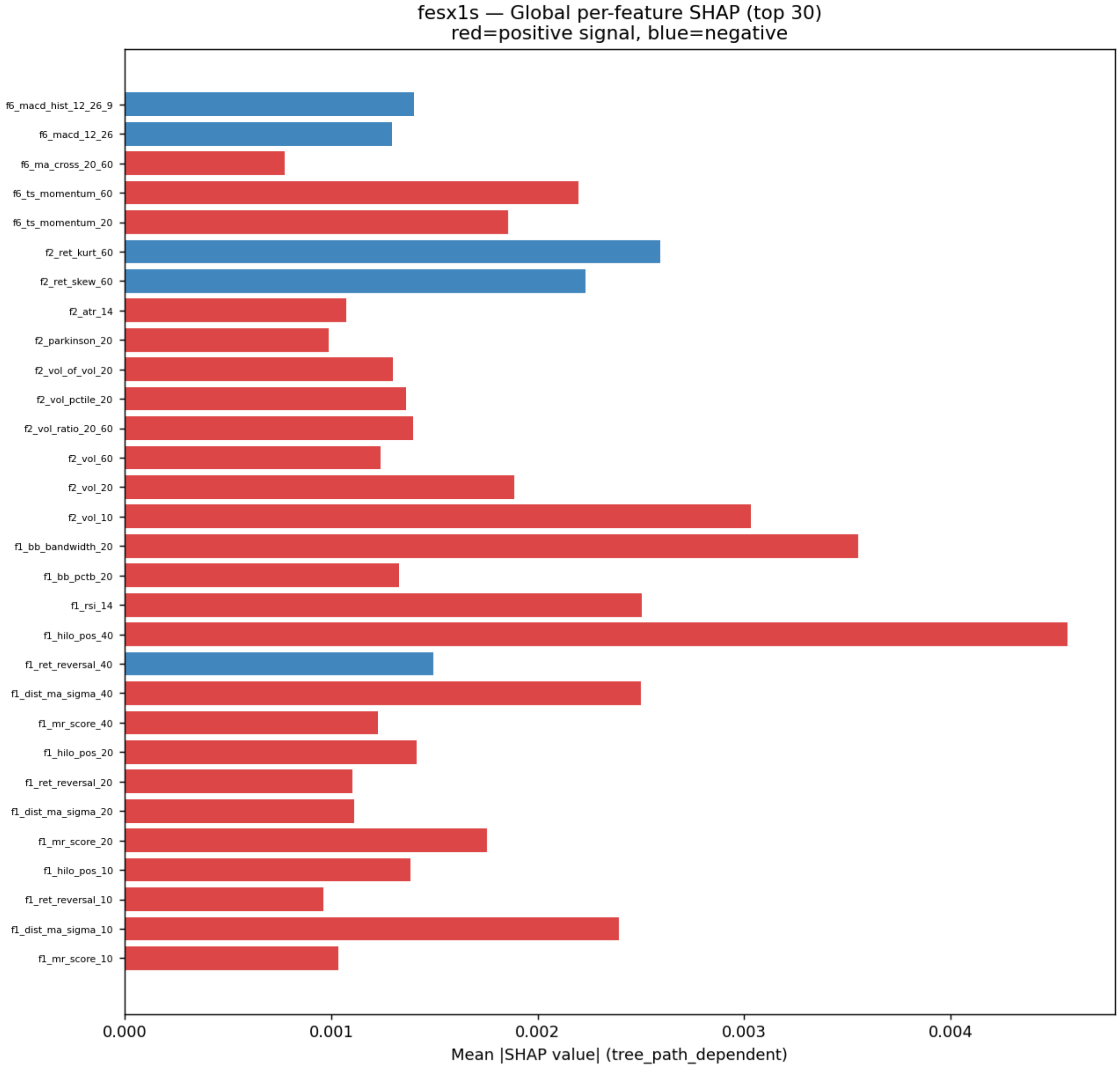
instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	sign
fesx1s	equity	fesx1s	tree	rf	0.4964	0.0242	0.4722	Fal





Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C2_f1	0.0061	0.0135	False
F5_signal	0.0039	0.0060	False
C15_f11	0.0030	0.0084	False
C7_f7	0.0029	0.0058	False
F4_latent	0.0023	0.0058	False
C1_f2	0.0017	0.0141	False
C5_f11	0.0004	0.0074	False
C13_f7	-0.0005	0.0027	False


fesx1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative
f6_macd_hist_12_26_9
f6_macd_12_26
f6_ma_cross_20_60
f6_ts_momentum_60
f6_ts_momentum_20
f2_ret_kurt_60
f2_ret_skew_60
f2_atr_14
f2_parkinson_20
f2_vol_of_vol_20
f2_vol_pctile_20
f2_vol_ratio_20_60
f2_vol_60
f2_vol_20
f2_vol_10
f1_bb_bandwidth_20
f1_bb_pctb_20
f1_rsi_14
f1_hilo_pos_40
f1_ret_reversal_40
f1_dist_ma_sigma_40
f1_mr_score_40
f1_hilo_pos_20
f1_ret_reversal_20
f1_dist_ma_sigma_20
f1_mr_score_20
f1_hilo_pos_10
f1_ret_reversal_10
f1_dist_ma_sigma_10
f1_mr_score_10
0.000
0.001
0.002
0.003
0.004
Mean |SHAP value| (tree_path_dependent)

Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
f1_hilo_pos_40	0.0046	0.0005	0.0190
f10_hl_range	0.0044	0.0002	0.0208
f11_vix_5d_change	0.0043	0.0005	0.0257
f4_pc1	0.0036	0.0009	0.0203
f1_bb_bandwidth_20	0.0036	0.0010	0.0141
f11_baltic_5d_change	0.0035	-0.0005	0.0184
hmm_vol_entropy	0.0034	0.0008	0.0230
f11_vix_level_z	0.0032	0.0011	0.0181
f7_volume_z_20	0.0032	0.0000	0.0167
f2_vol_10	0.0030	0.0001	0.0190
f10_oto_ret	0.0029	0.0003	0.0190
f7_volume_trend_20	0.0027	-0.0000	0.0141
f2_ret_kurt_60	0.0026	-0.0003	0.0130
f15_asset_class_dispersion_z	0.0025	0.0001	0.0169
f1_rsi_14	0.0025	0.0003	0.0133

7. Strategy — pooled portfolio and per-class attribution

There is a single backtest, not one per class. The live strategy is one volatility-targeted, equally-weighted book over all eleven contracts: an EWMA(60) volatility estimate scales each position to a 10% annualised target, leverage is capped, returns lag the weights by a day, and Grinold-Kahn costs (a 2bp half-spread plus 10bp of turnover) are charged. The conventions are recorded in `results/strategy_eval/eval_summary.md`. Method A follows every signal; Method B trades only when the calibrated metamodel probability exceeds one half.

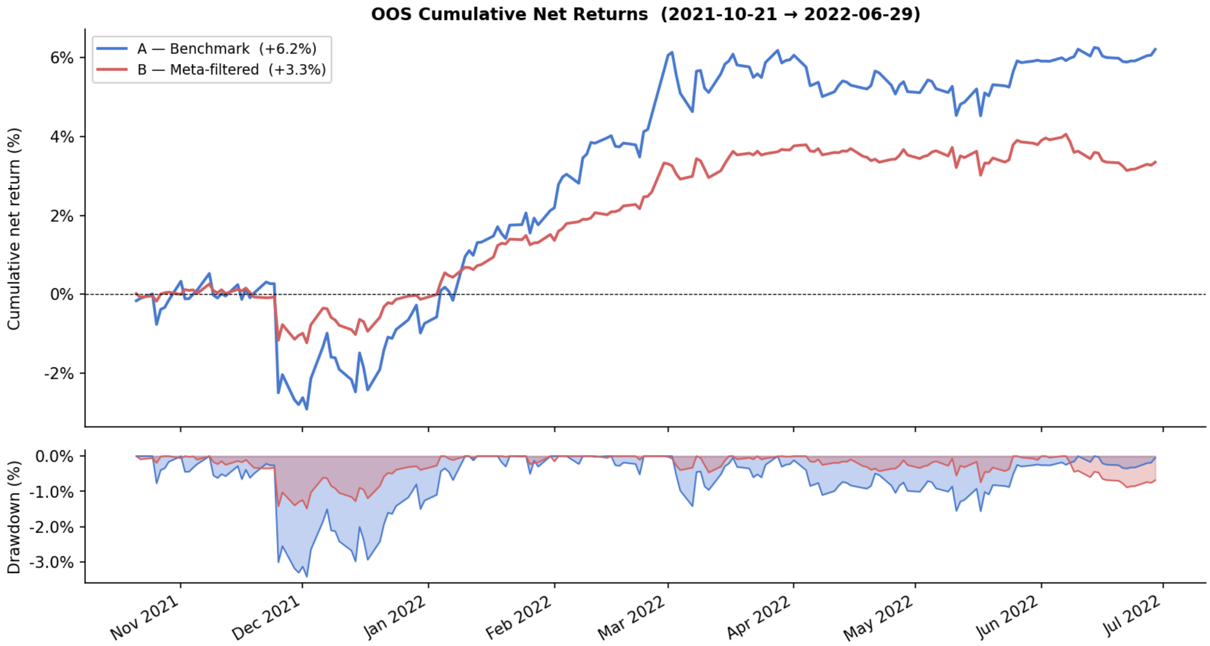
The headline table and cumulative-return figures below are the **pooled** result. The per-class table that follows is an **attribution** — this class's slice of the shared book — and not a standalone per-class Sharpe; with one pooled portfolio the instruments cannot be isolated into independent backtests.

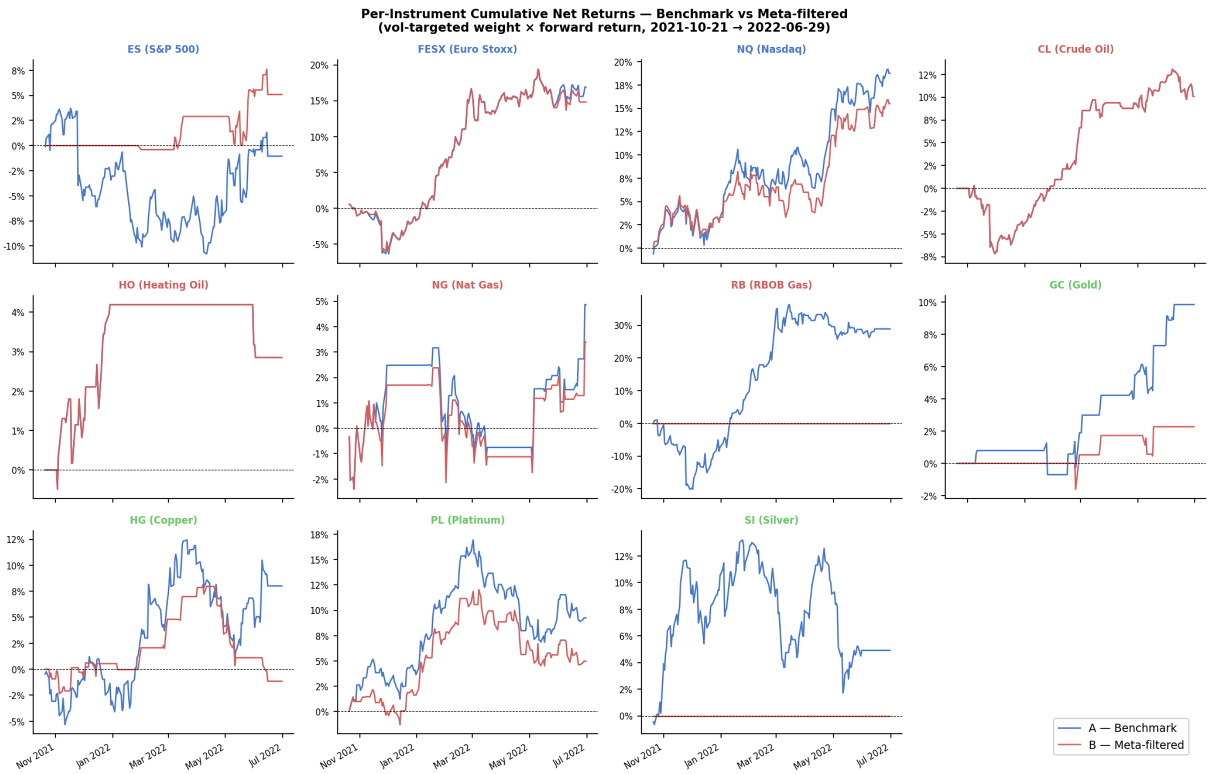
In [8]:
```
se = RESULTS / "strategy_eval"
table(load(se / "eval_summary.csv"), caption="Pooled portfolio — OOS headlir
show(se / "cumulative_returns.png", width=860)
show(se / "per_asset_cumulative_returns.png", width=860)
show(se / "per_instrument_contribution.png", width=860)
```

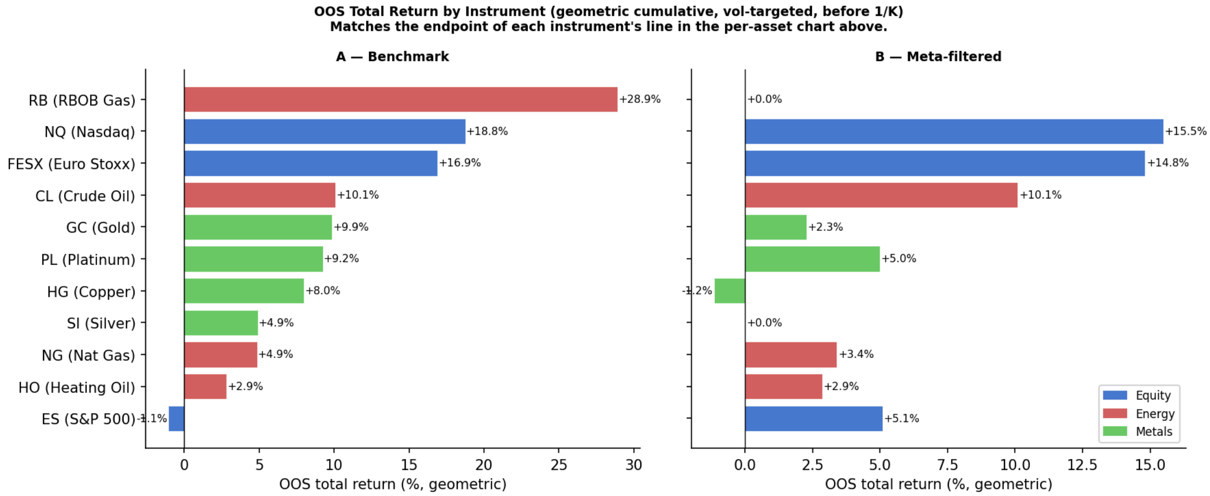
```python
# --- Per-class attribution within the pooled 1/K book (with numeric guards)
preds = load(OUT / "metamodel_predictions.csv")
assert not preds.empty, "metamodel_predictions.csv missing"
assert preds["calibrated_proba"].between(0, 1).all(), "calibrated_proba outs
assert set(INSTS) <= set(preds["instrument"].unique()), "class instruments a

cls = preds[preds["instrument"].isin(INSTS)].copy()
cls["taken"] = cls["calibrated_proba"] > 0.5
attr = (cls.groupby("instrument")
           .apply(lambda g: pd.Series({
               "n_events": len(g),
               "events_taken": int(g["taken"].sum()),
               "mean_calib_proba": g["calibrated_proba"].mean(),
               "label1_share": g["bin"].mean(),
               "hit_rate_taken": g.loc[g["taken"], "bin"].mean() if g["taker
           }), include_groups=False)
           .reindex(INSTS).reset_index())
assert (attr["events_taken"] <= attr["n_events"]).all(), "events_taken excee
table(attr, caption=f"{CLASS.title()} — attribution within the pooled book (
```

Pooled portfolio — OOS headline (eval_summary.csv)

Metric	A Benchmark	B all_or_nothing	B SOPS
Ann return	+8.62%	+4.65%	+4.65%
Ann vol	6.10%	2.88%	2.88%
Sharpe	1.412	1.614	1.614
t-stat	1.19	1.36	1.36
SR 95% CI low	-1.559	-1.286	-1.286
SR 95% CI high	3.750	3.821	3.821
PSR(SR*=0)	0.861	0.899	0.899
Max DD	-3.42%	-1.49%	-1.49%
Total cost (bps)	416.1	188.4	188.4
Annual turnover	48.28	21.86	21.86
Breakeven ½-spread	17.8 bps	21.3 bps	21.3 bps
Events taken	100%	54%	54%


OOS Cumulative Net Returns (2021-10-21 → 2022-06-29)
A — Benchmark (+6.2%)
B — Meta-filtered (+3.3%)
Cumulative net return (%)
Drawdown (%)
Nov 2021
Dec 2021
Jan 2022
Feb 2022
Mar 2022
Apr 2022
May 2022
Jun 2022
Jul 2022


Per-Instrument Cumulative Net Returns — Benchmark vs Meta-filtered
(vol-targeted weight × forward return, 2021-10-21 → 2022-06-29)
ES (S&P 500)
FESX (Euro Stoxx)
NQ (Nasdaq)
CL (Crude Oil)
HO (Heating Oil)
NG (Nat Gas)
RB (RBOB Gas)
GC (Gold)
HG (Copper)
PL (Platinum)
SI (Silver)
A — Benchmark
B — Meta-filtered



Equity — attribution within the pooled book (metamodel_predictions.csv)

instrument	n_events	events_taken	mean_calib_proba	label1_share	hit_rate_taken
es1s	150.0000	29.0000	0.4552	0.3933	0.4138
nq1s	171.0000	139.0000	0.5617	0.5439	0.5612
fesx1s	176.0000	166.0000	0.5352	0.5398	0.5361

8. Discussion

The honest equity read is one clear name, one tentative name and one pass. The Nasdaq metamodel is the result worth carrying forward. The S&P is plausible but should be treated as a hypothesis until a longer test confirms it. The Euro Stoxx offers no exploitable edge under the metamodel and would dilute the book if traded on conviction. None of this contradicts the signal being real; it says the filter adds value unevenly across the complex.

References

- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ, Wiley. [Triple-barrier labelling ch. 3; average uniqueness ch. 4; purged and combinatorial cross-validation ch. 7.]
- Project methodology notes (this repository): `reports/harry/00-context.md`, `01-signal-direction.md`, `02-labels.md`, `03-features.md`, `03-6-macro-features.md`, and `reports/missing-data-report.md`.
- Strategy conventions: `results/strategy_eval/eval_summary.md` (StrategyWeights lecture, slides 38–43).
- Course materials: *Systematic Trading Strategies with Machine Learning* (T3.03), lectures L1–L8.